작성 예시 · 1차시 · 경사하강법 · 이 정도 분량과 말투로 쓰면 돼요. 내용은 꼭 내 과제로 바꿔 써요.

# 심화 과제 보고서

| 학번 | 10000 | 이름 | 김예시 |
|---|---|---|---|
| 과제 제목 | 경사하강법으로 직선 찾기 | 작성일 | 2026. 11. 12. |

## 1. 무엇을 만들었나요? 문제를 내 말로 2~3 줄

공부 시간으로 시험 점수를 예측하는 직선 '점수 = W × 시간 + b'를 컴퓨터가 스스로 찾게 만들었다.
처음에는 W=0, b=0 으로 시작해서, 틀린 만큼 W 와 b 를 조금씩 고치는 일을 2000 번 반복한다.

## 2. 어떻게 나눠서 풀었나요? 단계마다 한 줄씩, 한국어로

| 단계 | 한 일 | 사용한 문법 |
|---|---|---|
| 1 | 공부 시간과 점수를 리스트 두 개에 넣고, W 와 b 를 0 으로 시작했다. | 리스트, 변수 |
| 2 | 학생마다 '예측 - 실제'(오차)를 구하고, W 와 b 를 어느 쪽으로 고칠지 모두 더했다. 오차의 제곱 평균도 같이 구했다. | for 문, 사칙연산 |
| 3 | W 와 b 를 오차의 반대 방향으로 학습률(0.01)만큼 옮기고, 이것을 2000 번 반복했다. | 이중 for 문, -= |
| 4 | 1, 10, 100, 2000 번째에 W, b, 오차를 출력해서 오차가 줄어드는지 확인했다. | if 문, f-string |

## 3. 핵심 코드와 실행 결과 가장 중요한 부분만, 왜 중요한지 한 줄

**핵심 코드**

```
x = [1, 2, 3, 4, 5]          # 공부 시간
y = [52, 61, 68, 79, 90]     # 점수
W, b = 0, 0
lr = 0.01                    # 학습률(보폭)

for step in range(1, 2001):
    dW, db, mse = 0, 0, 0
    for i in range(5):
        err = (W * x[i] + b) - y[i]
        dW += err * x[i]
        db += err
        mse += err ** 2 / 5
    W -= lr * dW / 5         # 반대 방향으로 조금 이동
    b -= lr * db / 5
    if step in (1, 10, 100, 2000):
        print(f"{step}번째: W={W:.2f}, b={b:.2f}, 오차
={mse:.2f}")
```

**실행 결과**

```
1 번째:
 W=2.29, b=0.70
 오차=5078.00

10 번째:
 W=13.78, b=4.48
 오차=736.60

100 번째:
 W=17.91, b=11.06
 오차=173.83

2000 번째:
 W=9.74, b=40.57
 오차=1.56
```

이 코드가 중요한 이유: W -= lr * dW / 5 줄. 오차가 커지는 방향의 반대로 조금씩 움직여서 오차가 5078 에서 1.56 까지 줄었다.

## 4. 막혔던 곳과 해결한 방법 오류 이름과 고친 방법

빨리 끝내려고 학습률을 0.3 으로 키웠더니 100 번째에 W 가 -854099...처럼 엄청 커지다가 OverflowError(숫자가 너무 큼)가 났다.
보폭이 너무 커서 골짜기를 넘어 반대편으로 계속 튕겨 나간 것이었다. 0.01 로 줄이니 오차가 매번 작아졌다.

## 5. 배운 점과 더 해 보고 싶은 것 새로 알게 된 것 하나, 다음에 해 볼 것 하나

AI 의 '학습'은 오차를 계산하고 반대 방향으로 조금 고치기를 반복하는 것이었다. 100 번째에는 W 가 17 까지 갔다가 다시 9.74 로 돌아오는 것이 신기했다.
다음에는 반복 횟수를 바꿔 가며 오차 그래프를 matplotlib 으로 그려 보고 싶다.

스스로 확인 ■ 자동 확인을 모두 통과했어요 ■ 코드를 직접 입력했어요 ■ 도움받은 곳: (힌트 2 단계, 1 차시 시뮬레이터)